Meeting Notes

Feb 19, 2026

BID controller

VM lifetime -> future knowledge

Feb 12, 2026

Octopus paper: Octopus_NSDI26_Feb11_2026.pdf

MPD: multi-ported CXL device

Fully-connected MPD pods have poor memory pooling savings:

- 4-port MPD = 4 servers in each pod
- Pooling across 4 servers are not enough (big gap between average and peak)

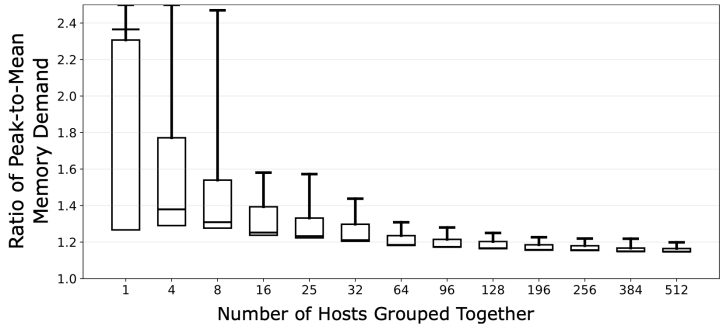
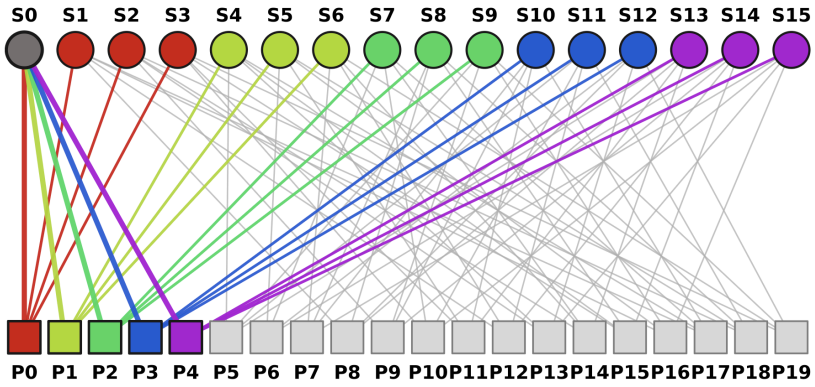


Figure 5: Ratio of peak memory demand to average memory demand across groups of servers of different sizes show that we need to group large numbers of servers to reduce outliers. Even groups of 25-32 servers, we would still need about 1.5× memory capacity to handle peaks. This data is for VM traces from Azure, gathered from ten production clusters over two weeks.

Octopus: Cost-effective CXL pods (use MPDs rather than switches) with good pooling savings:

- Servers (S0-S15) and MPDs (P0-P19) are sparsely connected
- Can be represented as a bipartite graph between servers and MPDs

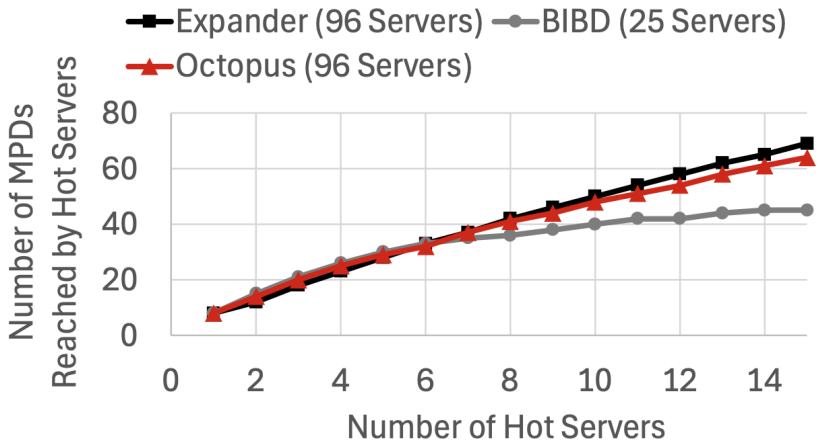


How to achieve higher pooling savings?

- Pooling savings are determined by the **peak memory usage across MPDs** because MPDs must be provisioned to handle the worst-case load
- Peak MPD usage is often dominated by **a small subset of "hot" servers** with high memory demand in a pod
- Better pooling savings => **the subset of hot servers should connect to more MPDs** (so that their their excess demand to be spread across as many distinct MPDs as possible)
    - This is know as **expansion**: For a given subset size k, we define the graph's expansion e_k as the minimum number of distinct MPDs connected to any subset of k servers

How to construct sparse bipartite graphs with good pooling savings?

- Best: large expander graphs (e.g., random graphs)

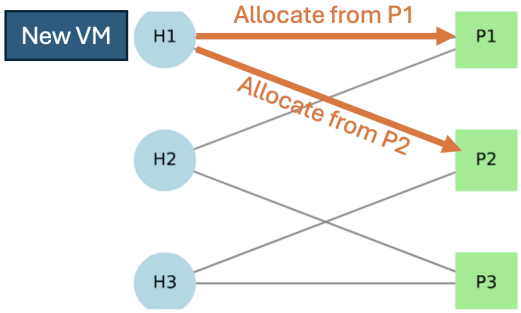


However, the real pooling is not a static, one-time allocation problem:

- Reality: VMs come and go constantly
- Challenge: We need to load balance memory usage across MPDs with dynamic server demand

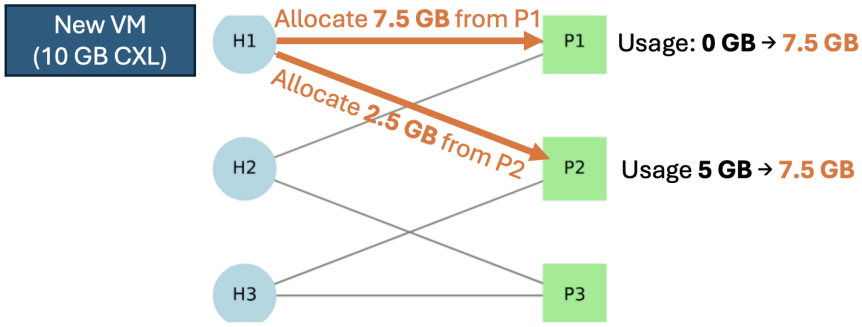
Memory allocation problem with Octopus:

- Each server connects to several MPDs
- When a new VM arrives on a server, from which MPDs should we allocate memory from?

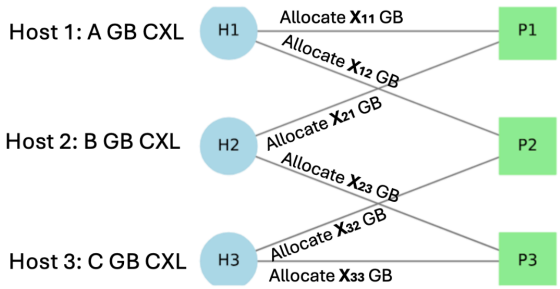


Memory allocation policies:

- Greedy:
  - Allocate from the least utilized MPDs
  - Split the allocation if necessary
  - Do not migrate allocated CXL memory between MPDs



- Optimal (impractical):
  - Allow immediately migrating allocated CXL memory between MPDs to balance load
  - Can be formulated and solved as a max-flow problem

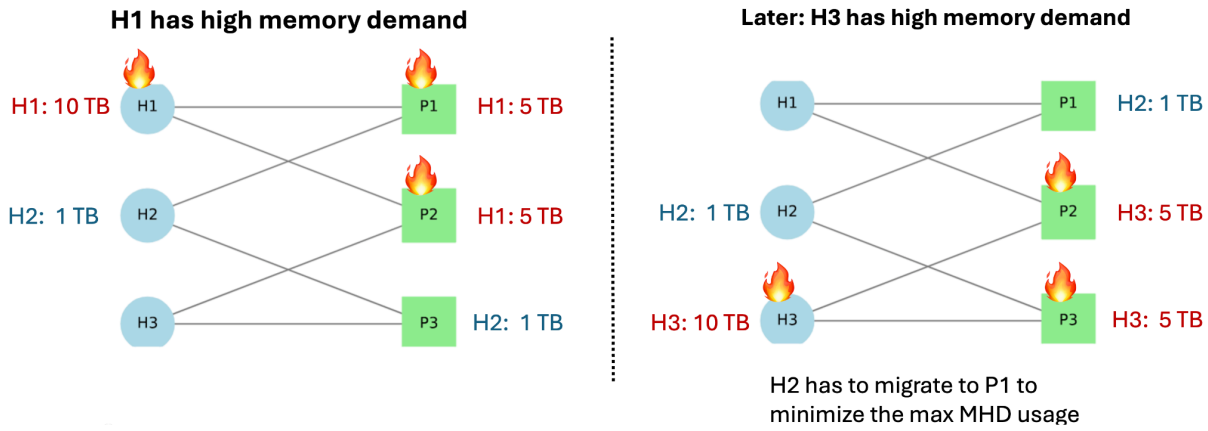


Becomes a convex optimization problem:

$$\min \quad \max(X_{11} + X_{21},\ X_{12} + X_{32},\ X_{23} + X_{33})$$

$$\text{s.t.} \quad X_{11} + X_{12} = A$$
$$X_{21} + X_{23} = B$$
$$X_{32} + X_{33} = C$$

Migration is required to attain optimal in some scenarios:



How to design better memory allocation policies?

- **Prediction**: Can we predict the per-server memory demand to guide memory allocation?
    - Possible signals: diurnal patterns, usage changing speed, CPU / disk utilization, VM lifetime
        - Per-customer prediction is easier
    - Can also consider simulating greedy with knowledge about the near future (e.g., 2 hours)
    - Relevant papers: Coach: Exploiting Temporal Patterns for All-Resource Oversubscription in Cloud Platforms
- **Migration**: If we can migrate allocated memory between MPDs, how does it improve pooling savings?
    - E.g., move 5% of VMs every 1 hour
- **Scheduling**: If we can migrate VM across servers, how does it improve pooling savings?
    - Make VM schedule aware of CXL pod topology and usage
- **RL**: Can we use RL to build a learned memory allocation algorithm?
    - Relevant papers: FleetIO: Managing Multi-Tenant Cloud Storage with Multi-Agent Reinforcement Learning
        - Using synthetic data to train?
    - Variance Reduction for Reinforcement Learning in Input-Driven Environments

Other problems:

- Bandwidth-aware allocation: balance both memory capacity and bandwidth usage
    - Every VM has memory bandwidth requirement
- Worst-case vs. average-case: how to test the worst-case savings?

VM Traces

VM traces from Azure

Each trace has a collection of VMs, each with:

- a **start time** and **end time** (when the VM is alive)
- a **memory footprint** (treated as a fixed amount for the VM's lifetime)
- the **server (host)** the VM is assigned to

The traces also have **each server's physical memory capacity**

How it is used in simulation:

- Time is discretized into coarse steps (5-minute intervals)
- As time advances, the simulator "replays" VM start and end events
- When a VM starts, its memory footprint is added to the total memory demand of its host
- When a VM ends, that allocated memory is removed

At each time step, the simulator can compute:

- Per-server memory demand as the sum of memory footprints of all currently active VMs on that server
- How much memory to allocate from each connected CXL device

TODOs

- Run greedy and optimal simulations with the `AMS20PrdApp19-tround.sqlite` trace
  - Can reuse the existing `greedy_pooling_simulation` and `optimal_pooling_simulation` functions
  - `optimal_pooling_simulation` currently computes the exact optimal solution which is very time consuming; could think about how to get approximations
- Understand the gap (in terms of memory savings) between greedy and optimal
- **Prediction**: Can we predict the per-server memory demand to guide memory allocation?
- **Migration**: If we can migrate allocated memory between CXL devices, how does it improve memory savings?
- **RL**: Can we use RL to build a learned memory allocation algorithm?